

Class Activity – 2

15 - 7- 26

Scenarios	Programming Language
Scenario 1	Python, R
Scenario 2	Python, R
Scenario 3	Python, R
Scenario 4	Python, R
Scenario 5	Python, R
Scenario 6	Python, R
Scenario 7	Python, R
Scenario 8	Python, R
Scenario 9	Python, R
Scenario 10	Python, R

Scenario 1: Smart City Air Quality Monitoring

Python:

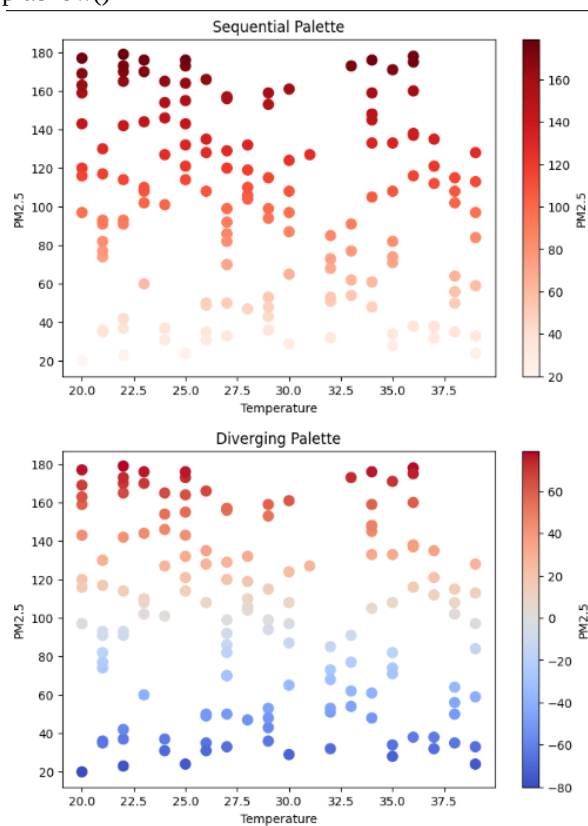
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(10)
n = 150
df = pd.DataFrame({
    "Location": range(1, n+1),
    "PM2.5": np.random.randint(20,180,n),
    "NO2": np.random.randint(10,90,n),
    "CO": np.random.uniform(0.5,6,n),
    "Temperature": np.random.randint(20,40,n),
    "Humidity": np.random.randint(30,90,n)
})
plt.figure(figsize=(8,5))
plt.scatter(df["Temperature"],
            df["PM2.5"],
            c=df["PM2.5"],
            cmap='Reds',
            s=70)
plt.colorbar(label="PM2.5")
plt.title("Sequential Palette")
plt.xlabel("Temperature")
plt.ylabel("PM2.5")
plt.show()
plt.figure(figsize=(8,5))
```

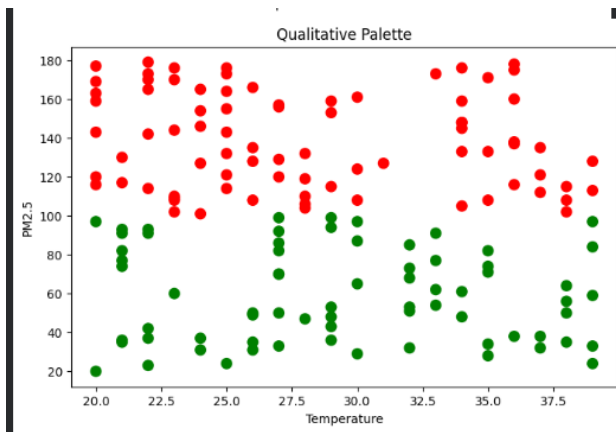
```
plt.scatter(df["Temperature"],
            df["PM2.5"],
            c=df["PM2.5"]-100,
            cmap='coolwarm',
            s=70)

plt.colorbar(label="PM2.5")
plt.title("Diverging Palette")
plt.xlabel("Temperature")
plt.ylabel("PM2.5")
plt.show()

plt.figure(figsize=(8,5))
colors=["red" if x>100 else "green" for x in df["PM2.5"]]
plt.scatter(df["Temperature"],
            df["PM2.5"],
            c=colors,
            s=70)

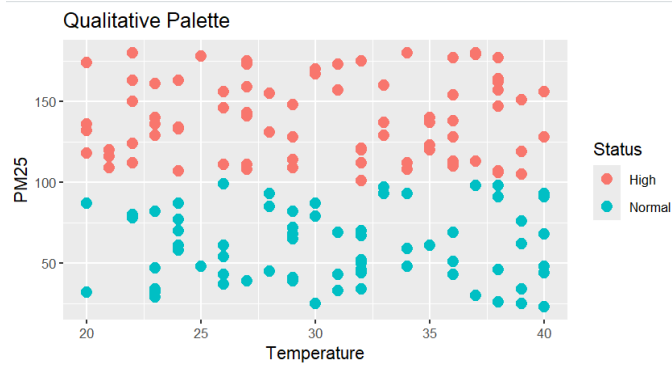
plt.title("Qualitative Palette")
plt.xlabel("Temperature")
plt.ylabel("PM2.5")
plt.show()
```





R Program:

```
library(ggplot2)
set.seed(10)
n=150
df=data.frame(
  Location=1:n,
  PM25=sample(20:180,n,replace=TRUE),
  NO2=sample(10:90,n,replace=TRUE),
  CO=runif(n,0.5,6),
  Temperature=sample(20:40,n,replace=TRUE),
  Humidity=sample(30:90,n,replace=TRUE)
)
ggplot(df,aes(Temperature,PM25,color=PM25))+
  geom_point(size=3)+
  scale_color_gradient(low="yellow",high="red")+
  ggtitle("Sequential Palette")
ggplot(df,aes(Temperature,PM25,color=PM25-100))+
  geom_point(size=3)+
  scale_color_gradient2(low="blue",
    mid="white",
    high="red",
    midpoint=0)+
  ggtitle("Diverging Palette")
df$Status=ifelse(df$PM25>100,"High","Normal")
ggplot(df,aes(Temperature,PM25,color=Status))+
  geom_point(size=3)+
  ggtitle("Qualitative Palette")
```

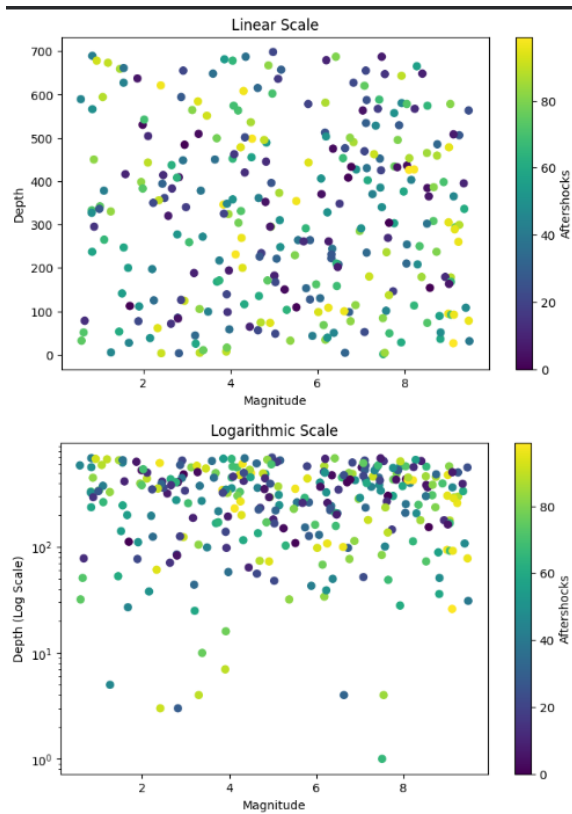


Scenario 2: Earthquake Risk Analysis

Python:

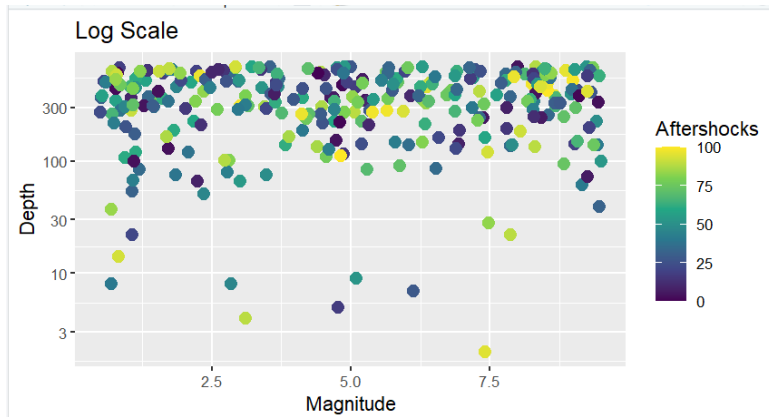
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(20)
n=300
df=pd.DataFrame({
    "Magnitude":np.random.uniform(0.5,9.5,n),
    "Depth":np.random.randint(1,700,n),
    "Aftershocks":np.random.randint(0,100,n)
})
plt.figure(figsize=(8,5))
plt.scatter(df["Magnitude"],
            df["Depth"],
            c=df["Aftershocks"],
            cmap="viridis")
plt.colorbar(label="Aftershocks")
plt.xlabel("Magnitude")
plt.ylabel("Depth")
plt.title("Linear Scale")
plt.show()
plt.figure(figsize=(8,5))
plt.scatter(df["Magnitude"],
            df["Depth"],
            c=df["Aftershocks"],
            cmap="viridis")

plt.yscale("log")
plt.colorbar(label="Aftershocks")
plt.xlabel("Magnitude")
plt.ylabel("Depth (Log Scale)")
plt.title("Logarithmic Scale")
plt.show()
```



R Program:

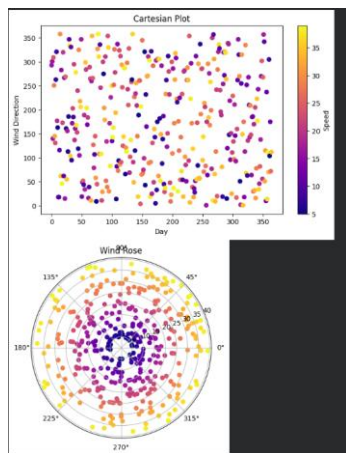
```
library(ggplot2)
set.seed(20)
n=300
df=data.frame(
  Magnitude=runif(n,0.5,9.5),
  Depth=sample(1:700,n,replace=TRUE),
  Aftershocks=sample(0:100,n,replace=TRUE)
)
ggplot(df,aes(Magnitude,Depth,color=Aftershocks))+
  geom_point(size=3)+
  scale_color_viridis_c()+
  ggtitle("Linear Scale")
ggplot(df,aes(Magnitude,Depth,color=Aftershocks))+
  geom_point(size=3)+
  scale_y_log10()+
  scale_color_viridis_c()+
  ggtitle("Log Scale")
```



Scenario 3: Wind Direction Analysis for Airport Operations

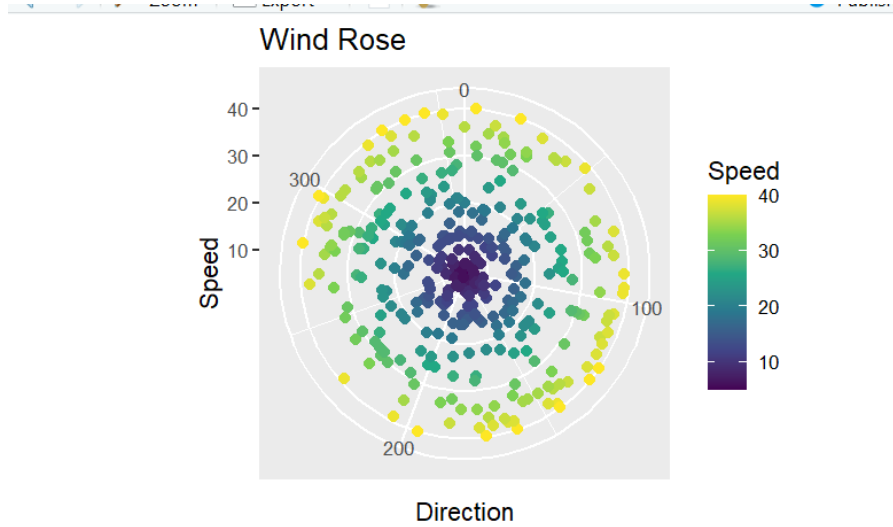
Python:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(30)
n=365
direction=np.random.randint(0,360,n)
speed=np.random.randint(5,40,n)
df=pd.DataFrame({
    "Direction":direction,
    "Speed":speed
})
plt.figure(figsize=(8,5))
plt.scatter(df.index,
            df["Direction"],
            c=df["Speed"],
            cmap="plasma")
plt.xlabel("Day")
plt.ylabel("Wind Direction")
plt.title("Cartesian Plot")
plt.colorbar(label="Speed")
plt.show()
theta=np.deg2rad(df["Direction"])
ax=plt.subplot(111,projection="polar")
ax.scatter(theta,
            df["Speed"],
            c=df["Speed"],
            cmap="plasma")
plt.title("Wind Rose")
plt.show()
```



R Program:

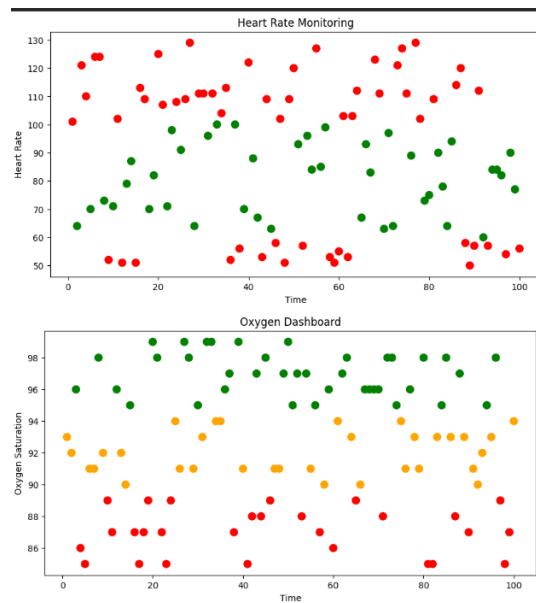
```
library(ggplot2)
set.seed(30)
n=365
df=data.frame(
  Day=1:n,
  Direction=sample(0:359,n,replace=TRUE),
  Speed=sample(5:40,n,replace=TRUE)
)
ggplot(df,aes(Day,Direction,color=Speed))+
  geom_point(size=2)+
  scale_color_viridis_c()+
  ggtitle("Cartesian Plot")
ggplot(df,aes(x=Direction,y=Speed,color=Speed))+
  geom_point(size=2)+
  coord_polar()+
  scale_color_viridis_c()+
  ggtitle("Wind Rose")
```



Scenario 4: Hospital Patient Monitoring Dashboard

Python:

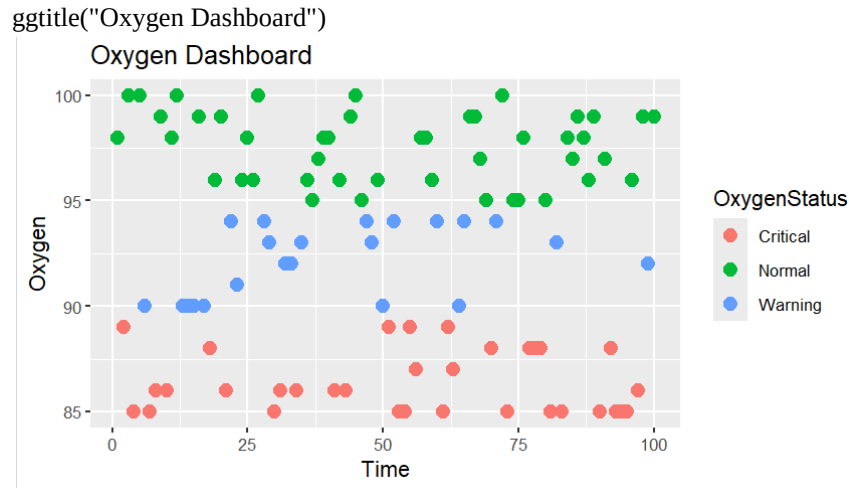
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
n = 100
df = pd.DataFrame({
    "Time": range(1, n + 1),
    "HeartRate": np.random.randint(50, 130, n),
    "Oxygen": np.random.randint(85, 100, n),
    "Temperature": np.random.uniform(36, 40, n),
    "Respiration": np.random.randint(10, 35, n)
})
# Highlight heart rate
colors = []
for hr in df["HeartRate"]:
    if hr < 60 or hr > 100:
        colors.append("red")
    else:
        colors.append("green")
plt.figure(figsize=(10,5))
plt.scatter(df["Time"],
            df["HeartRate"],
            c=colors,
            s=70)
plt.xlabel("Time")
plt.ylabel("Heart Rate")
plt.title("Heart Rate Monitoring")
plt.show()
# Oxygen Dashboard
colors=[]
for x in df["Oxygen"]:
    if x<90:
        colors.append("red")
    elif x<95:
        colors.append("orange")
    else:
        colors.append("green")
plt.figure(figsize=(10,5))
plt.scatter(df["Time"],
            df["Oxygen"],
            c=colors,
            s=70)
plt.xlabel("Time")
plt.ylabel("Oxygen Saturation")
plt.title("Oxygen Dashboard")
plt.show()
```

R Program:

```
library(ggplot2)
set.seed(42)
n=100
df=data.frame(
  Time=1:n,
  HeartRate=sample(50:130,n,replace=TRUE),
  Oxygen=sample(85:100,n,replace=TRUE),
  Temperature=runif(n,36,40),
  Respiration=sample(10:35,n,replace=TRUE)
)
df$HRStatus=ifelse(df$HeartRate<60 |
  df$HeartRate>100,
  "Critical",
  "Normal")

ggplot(df,
  aes(Time,
  HeartRate,
  color=HRStatus))+
  geom_point(size=3)+
  ggtitle("Heart Rate Dashboard")
df$OxygenStatus=ifelse(df$Oxygen<90,
  "Critical",
  ifelse(df$Oxygen<95,
  "Warning",
  "Normal"))
ggplot(df,
  aes(Time,
  Oxygen,
  color=OxygenStatus))+
  geom_point(size=3)+
```



Scenario 5: Global Climate Change Study

Python:

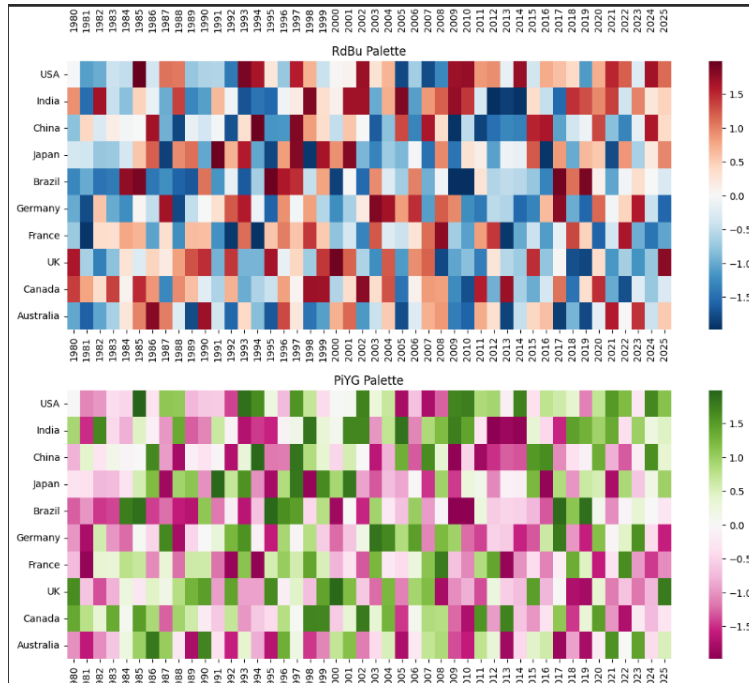
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
np.random.seed(50)
countries = ["USA", "India", "China", "Japan", "Brazil",
             "Germany", "France", "UK", "Canada", "Australia"]
years = list(range(1980, 2026))
data = np.random.uniform(-2, 2,
                          (len(countries),
                           len(years)))
df = pd.DataFrame(data,
                  index=countries,
                  columns=years)

# Palette 1
plt.figure(figsize=(14, 5))
sns.heatmap(df,
            cmap="coolwarm",
            center=0)
plt.title("Coolwarm Palette")
plt.show()

# Palette 2
plt.figure(figsize=(14, 5))
sns.heatmap(df,
            cmap="RdBu_r",
            center=0)

plt.title("RdBu Palette")
plt.show()
```

```
# Palette 3
plt.figure(figsize=(14,5))
sns.heatmap(df,
            cmap="PiYG",
            center=0)
plt.title("PiYG Palette")
plt.show()
```



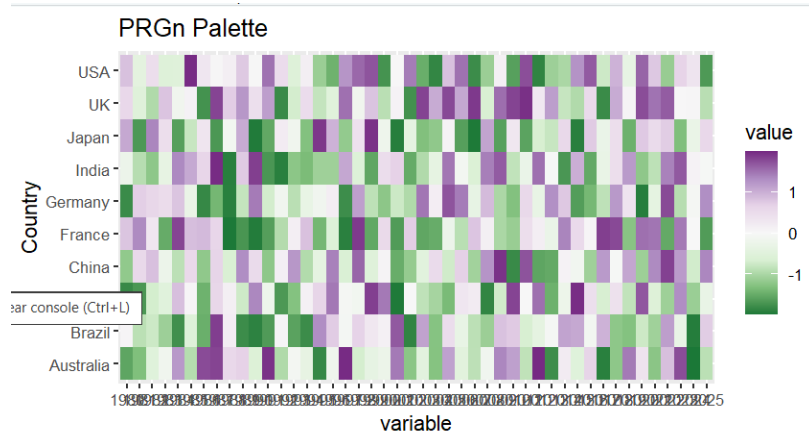
R Program:

```
library(ggplot2)
library(reshape2)
set.seed(50)
countries=c("USA",
            "India",
            "China",
            "Japan",
            "Brazil",
            "Germany",
            "France",
            "UK",
            "Canada",
            "Australia")
years=1980:2025
mat=matrix(runif(length(countries)*length(years),-2, 2),
nrow=length(countries))
df=as.data.frame(mat)
colnames(df)=years
df$Country=countries
long=melt(df,
id.vars="Country")
```

```

ggplot(long,
aes(variable,
Country,
fill=value))+
geom_tile()+
scale_fill_gradient2(low="blue",
mid="white",
high="red",
midpoint=0)+
ggtitle("Coolwarm Style")
ggplot(long,
aes(variable,
Country,
fill=value))+
geom_tile()+
scale_fill_distiller(palette="RdBu")+
ggtitle("RdBu Palette")
ggplot(long,
aes(variable,
Country,
fill=value))+
geom_tile()+
scale_fill_distiller(palette="PRGn")+
ggtitle("PRGn Palette")

```



Scenario 6: E-Commerce Sales Analytics

Python:

```

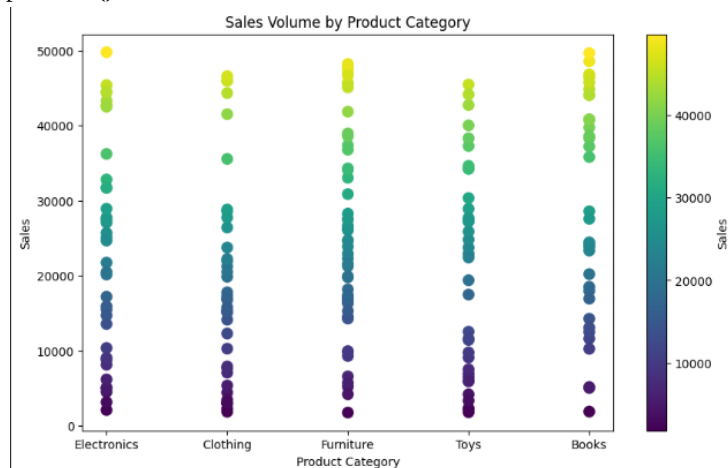
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(100)
categories = ["Electronics", "Clothing", "Furniture", "Toys", "Books"]
regions = ["North", "South", "East", "West", "Central"]

```

```

n = 200
df = pd.DataFrame({
    "Category": np.random.choice(categories, n),
    "Region": np.random.choice(regions, n),
    "Sales": np.random.randint(1000,50000,n),
    "Profit": np.random.randint(100,10000,n)
})
# Assign numeric values for plotting
cat_num = pd.factorize(df["Category"])[0]
plt.figure(figsize=(10,6))
scatter = plt.scatter(
    cat_num,
    df["Sales"],
    c=df["Sales"],
    cmap="viridis",
    s=80
)
plt.xticks(range(len(categories)), categories)
plt.xlabel("Product Category")
plt.ylabel("Sales")
plt.title("Sales Volume by Product Category")
plt.colorbar(scatter,label="Sales")
plt.show()

```



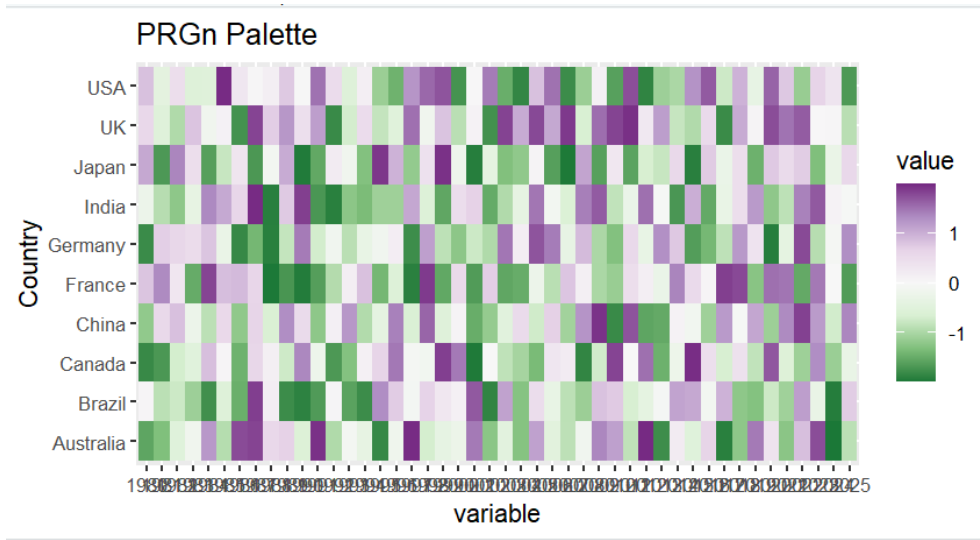
R Program:

```

library(ggplot2)
set.seed(100)
categories <- c("Electronics","Clothing","Furniture","Toys","Books")
regions <- c("North","South","East","West","Central")
n <- 200
df <- data.frame(
  Category = sample(categories,n,replace=TRUE),
  Region = sample(regions,n,replace=TRUE),
  Sales = sample(1000:50000,n,replace=TRUE),
  Profit = sample(100:10000,n,replace=TRUE)
)

```

```
ggplot(df,
aes(Category,
Sales,
color=Sales))+
geom_point(size=3)+
scale_color_viridis_c()+
ggtitle("Sales Analytics")
```



Scenario 7: Satellite Image Vegetation Analysis

Python:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
np.random.seed(200)
```

```
rows=20
cols=20
```

```
ndvi=np.random.uniform(-1,1,(rows,cols))
```

```
plt.figure(figsize=(6,6))
plt.imshow(ndvi,
cmap="Greens")
```

```
plt.colorbar(label="NDVI")
```

```
plt.title("Sequential Palette")
plt.show()
```

```
plt.figure(figsize=(6,6))
```

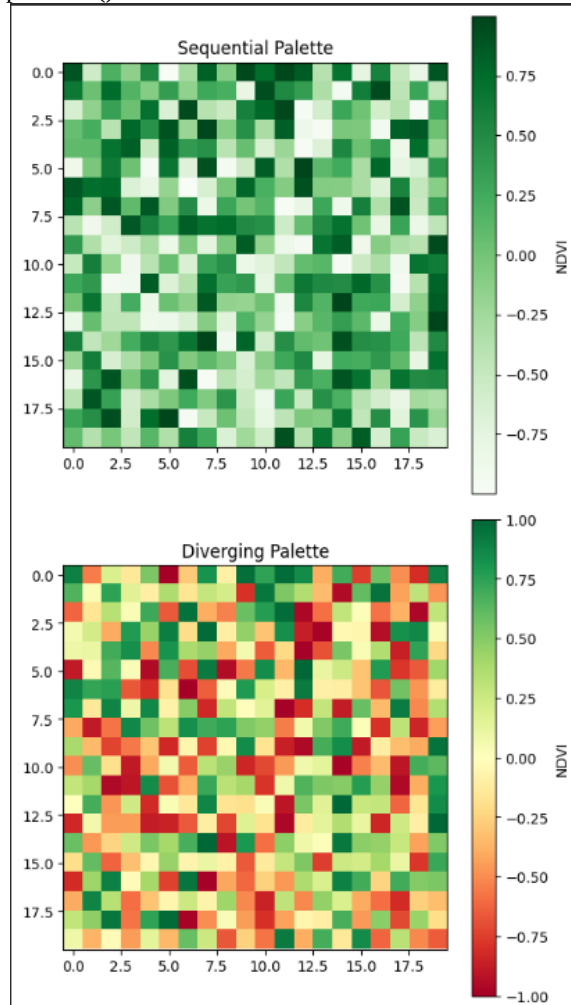
```
plt.imshow(ndvi,
cmap="RdYlGn",
vmin=-1,
```

```
vmax=1)
```

```
plt.colorbar(label="NDVI")
```

```
plt.title("Diverging Palette")
```

```
plt.show()
```



R Program:

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Generate sample data
```

```
dates = pd.date_range("2024-01-01", periods=100)
```

```
df = pd.DataFrame({
```

```
    "Date": dates,
```

```
    "Bitcoin": np.random.uniform(30000, 70000, 100),
```

```
    "Ethereum": np.random.uniform(1000, 5000, 100),
```

```
    "Solana": np.random.uniform(20, 250, 100),
```

```

"Cardano": np.random.uniform(0.2, 2.5, 100)
})

# Linear Scale
plt.figure(figsize=(10,5))
plt.plot(df["Date"], df["Bitcoin"], label="Bitcoin")
plt.plot(df["Date"], df["Ethereum"], label="Ethereum")
plt.plot(df["Date"], df["Solana"], label="Solana")
plt.plot(df["Date"], df["Cardano"], label="Cardano")

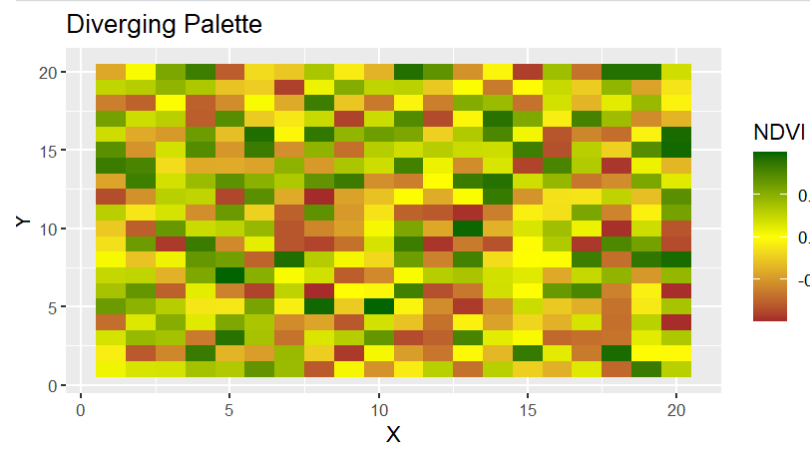
plt.title("Cryptocurrency Prices (Linear Scale)")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.xticks(rotation=45)
plt.show()

# Log Scale
plt.figure(figsize=(10,5))
plt.plot(df["Date"], df["Bitcoin"], label="Bitcoin")
plt.plot(df["Date"], df["Ethereum"], label="Ethereum")
plt.plot(df["Date"], df["Solana"], label="Solana")
plt.plot(df["Date"], df["Cardano"], label="Cardano")

plt.yscale("log")

plt.title("Cryptocurrency Prices (Log Scale)")
plt.xlabel("Date")
plt.ylabel("Price (Log)")
plt.legend()
plt.xticks(rotation=45)
plt.show()

```



Scenario 8: Cryptocurrency Market Volatility

Python:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Generate sample data
dates = pd.date_range("2024-01-01", periods=100)

df = pd.DataFrame({
    "Date": dates,
    "Bitcoin": np.random.uniform(30000, 70000, 100),
    "Ethereum": np.random.uniform(1000, 5000, 100),
    "Solana": np.random.uniform(20, 250, 100),
    "Cardano": np.random.uniform(0.2, 2.5, 100)
})

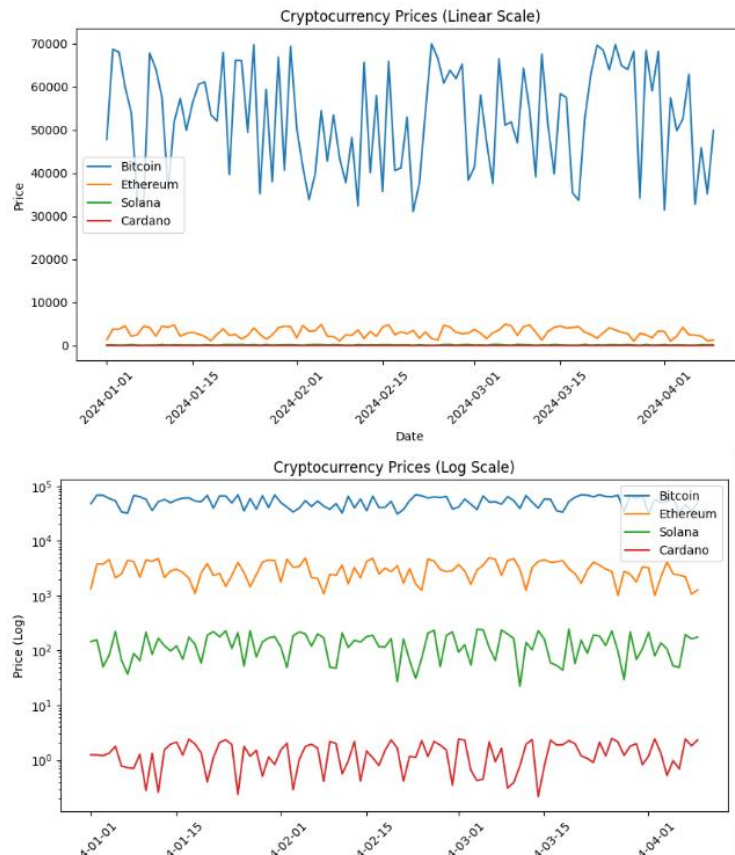
# Linear Scale
plt.figure(figsize=(10,5))
plt.plot(df["Date"], df["Bitcoin"], label="Bitcoin")
plt.plot(df["Date"], df["Ethereum"], label="Ethereum")
plt.plot(df["Date"], df["Solana"], label="Solana")
plt.plot(df["Date"], df["Cardano"], label="Cardano")

plt.title("Cryptocurrency Prices (Linear Scale)")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.xticks(rotation=45)
plt.show()

# Log Scale
plt.figure(figsize=(10,5))
plt.plot(df["Date"], df["Bitcoin"], label="Bitcoin")
plt.plot(df["Date"], df["Ethereum"], label="Ethereum")
plt.plot(df["Date"], df["Solana"], label="Solana")
plt.plot(df["Date"], df["Cardano"], label="Cardano")

plt.yscale("log")

plt.title("Cryptocurrency Prices (Log Scale)")
plt.xlabel("Date")
plt.ylabel("Price (Log)")
plt.legend()
plt.xticks(rotation=45)
plt.show()
```



R Program:

```
library(ggplot2)
```

```
library(tidyr)
```

```
set.seed(100)
```

```
dates <- seq(as.Date("2024-01-01"), by="day", length.out=100)
```

```
df <- data.frame(
  Date=dates,
  Bitcoin=runif(100,30000,70000),
  Ethereum=runif(100,1000,5000),
  Solana=runif(100,20,250),
  Cardano=runif(100,0.2,2.5)
)
```

```
long <- pivot_longer(df,
  cols=-Date,
  names_to="Coin",
  values_to="Price")
```

```
# Linear Scale
ggplot(long,
```

```

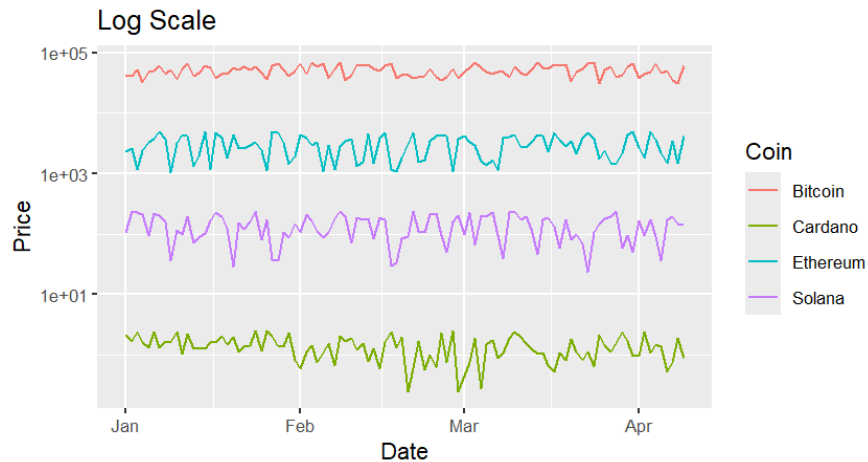
aes(Date,Price,color=Coin))+
geom_line()+
ggtitle("Linear Scale")

```

```

# Log Scale
ggplot(long,
aes(Date,Price,color=Coin))+
geom_line()+
scale_y_log10()+
ggtitle("Log Scale")

```



Scenario 9: Marathon Runner Performance Analysis

Python:

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

```

```

np.random.seed(123)

```

```

n = 200

```

```

df = pd.DataFrame({
    "Distance": np.linspace(0,42,n),
    "Speed": np.random.uniform(8,18,n),
    "HeartRate": np.random.randint(90,190,n),
    "Direction": np.random.randint(0,360,n)
})

```

```

# Cartesian Plot
plt.figure(figsize=(10,5))

```

```

plt.scatter(df["Distance"],
            df["Speed"],
            c=df["Speed"],

```

```

cmap="viridis")

plt.colorbar(label="Speed")

plt.xlabel("Distance (km)")
plt.ylabel("Speed (km/h)")
plt.title("Runner Speed")
plt.show()

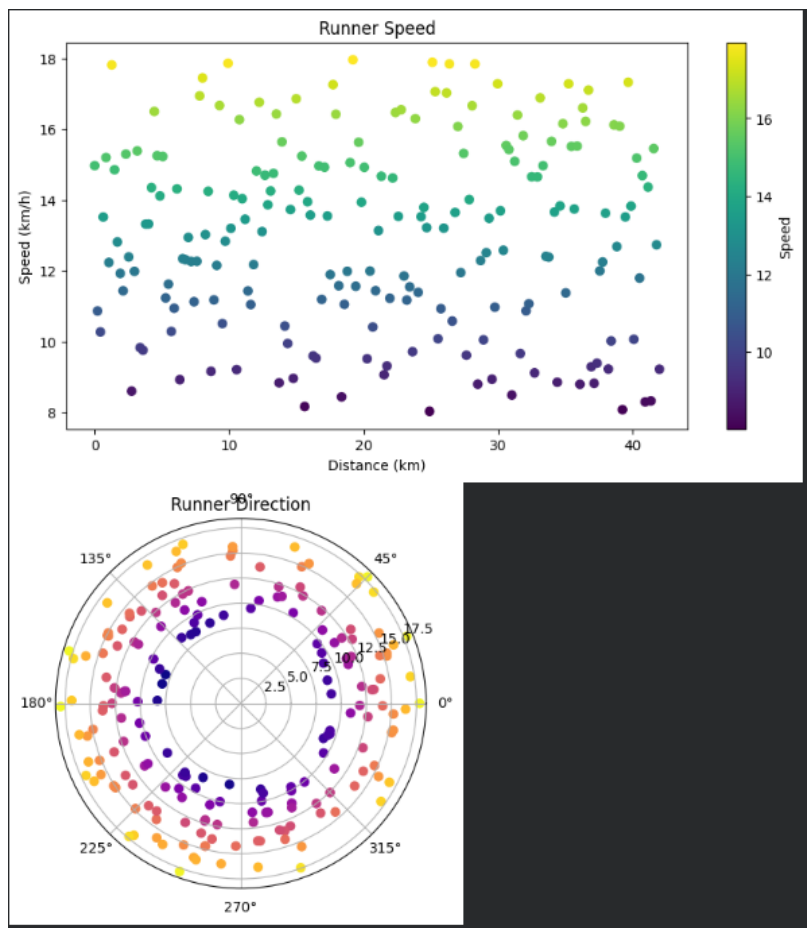
# Polar Plot
theta=np.deg2rad(df["Direction"])

ax=plt.subplot(111,projection="polar")

ax.scatter(theta,
           df["Speed"],
           c=df["Speed"],
           cmap="plasma")

plt.title("Runner Direction")
plt.show()

```



R Program:

```
library(ggplot2)
```

```
set.seed(123)
```

```
n=200
```

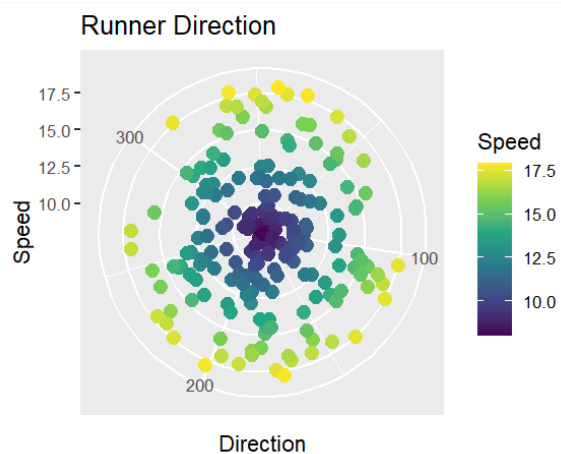
```
df=data.frame(  
  Distance=seq(0,42,length.out=n),  
  Speed=runif(n,8,18),  
  HeartRate=sample(90:190,n,replace=TRUE),  
  Direction=sample(0:359,n,replace=TRUE)  
)
```

```
# Cartesian Plot
```

```
ggplot(df,  
  aes(Distance,  
    Speed,  
    color=Speed))+  
  geom_point(size=3)+  
  scale_color_viridis_c()+  
  ggtitle("Runner Speed")
```

```
# Polar Plot
```

```
ggplot(df,  
  aes(Direction,  
    Speed,  
    color=Speed))+  
  geom_point(size=3)+  
  coord_polar()+  
  scale_color_viridis_c()+  
  ggtitle("Runner Direction")
```



Scenario 10: COVID-19 Vaccination Campaign Dashboard

Python:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Sample Data
np.random.seed(101)

districts = ["North", "South", "East", "West", "Central"]
age_groups = ["0-18", "19-40", "41-60", "60+"]
vaccines = ["Covaxin", "Covishield", "Sputnik"]

n = 100

df = pd.DataFrame({
    "District": np.random.choice(districts, n),
    "AgeGroup": np.random.choice(age_groups, n),
    "Vaccine": np.random.choice(vaccines, n),
    "VaccinationRate": np.random.randint(40, 101, n)
})

# -----
# Dashboard 1 : Policymaker
# -----

plt.figure(figsize=(8,5))

district_avg = df.groupby("District")["VaccinationRate"].mean()

bars = plt.bar(
    district_avg.index,
    district_avg.values,
    color=["red", "orange", "yellow", "green", "blue"]
)

plt.ylabel("Average Vaccination Rate")
plt.title("Policymaker Dashboard")
plt.show()

# -----
# Dashboard 2 : Public Dashboard
# -----

plt.figure(figsize=(8,5))

colors=[]
```

```

for rate in district_avg.values:
    if rate>=80:
        colors.append("green")
    elif rate>=60:
        colors.append("gold")
    else:
        colors.append("red")

```

```

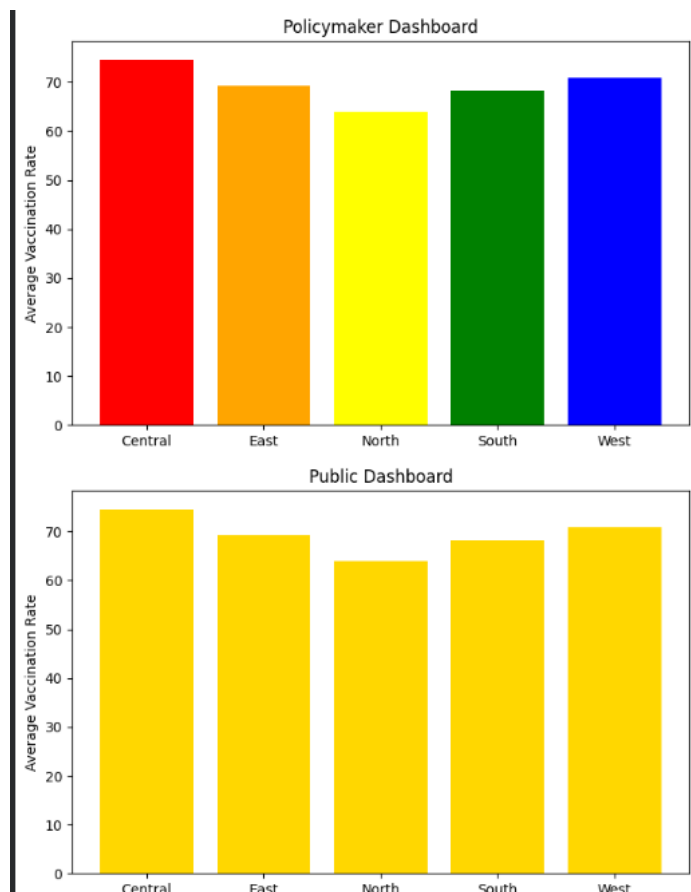
plt.bar(
    district_avg.index,
    district_avg.values,
    color=colors
)

```

```

plt.ylabel("Average Vaccination Rate")
plt.title("Public Dashboard")
plt.show()

```



R Program:

```

library(ggplot2)
library(dplyr)

```

```

set.seed(101)

districts <- c("North", "South", "East", "West", "Central")
age <- c("0-18", "19-40", "41-60", "60+")
vaccines <- c("Covaxin", "Covishield", "Sputnik")

n <- 100

df <- data.frame(
  District = sample(districts, n, replace=TRUE),
  AgeGroup = sample(age, n, replace=TRUE),
  Vaccine = sample(vaccines, n, replace=TRUE),
  VaccinationRate = sample(40:100, n, replace=TRUE)
)

summary_df <- df %>%
  group_by(District) %>%
  summarise(Average = mean(VaccinationRate))

# -----
# Policymaker Dashboard
# -----

ggplot(summary_df,
  aes(District,
    Average,
    fill=Average))+
  geom_col()+
  scale_fill_gradient(
    low="yellow",
    high="darkgreen"
  )+
  ggtitle("Policymaker Dashboard")

# -----
# Public Dashboard
# -----

summary_df$Status <- ifelse(summary_df$Average >= 80,
  "High",
  ifelse(summary_df$Average >= 60,
    "Medium",
    "Low"))

ggplot(summary_df,
  aes(District,
    Average,

```



```
fill=Status))+  
geom_col()+  
ggtitle("Public Dashboard")
```

